# Phase 1B — Auth0 Pre-Connection Dossier (CAUL AI)

**Status:** PRE-CONNECTION PLANNING ONLY. No Auth0 connection, no external calls, no code wired, no spend. This documents exactly what is required so that, upon your written approval + the values in §7, the identity cutover is a fast, low-risk change. Built from the current Auth0 integration playbook (latest SDKs).

## 0. Non-negotiables (carried from Phase 1A)

- **RBAC stays in PostgreSQL.** Auth0 provides *authentication/identity only*; roles & permissions are still resolved from our `roles`/`permissions` tables. We never trust roles from Auth0 claims.
- **Keep the labeled stub until cutover.** `StubIdentityProvider` remains the default; Auth0 is added behind the existing `IdentityProvider` interface and enabled by a feature flag — instant rollback.
- **Separate Auth0 tenants per environment** (dev/staging/prod). No shared tenant.
- **No client secret in the SPA.** React is a public client using Authorization Code + PKCE.
- **Owner identity:** the ExecOwner account maps to **sean.gaden@gmail.com** (real owner), keyed by Auth0 `sub`.

## 1. Auth0 resources to create (per environment)

| Resource | Where | Produces |
|---|---|---|
| Tenant/domain | Dashboard -> Settings -> General | `AUTH0_DOMAIN` e.g. `cauldron-staging.us.auth0.com` |
| Application (SPA) | Applications -> Create -> **Single Page Application** | `AUTH0_CLIENT_ID` (public) |
| API / resource server | Applications -> APIs -> Create (RS256) | **Identifier = `AUTH0_AUDIENCE`** e.g. `https://api.caul.cauldron` |
| Allowlists on the SPA | SPA -> Settings | Callback / Logout / Web Origins = our frontend origin |

Allowlists (no wildcards): set Callback URL, Logout URL, and Web Origin to the CAUL AI frontend origin (staging preview URL first; production domain later). A dedicated `/callback` route is optional.

## 2. Keys / env vars CAUL AI will consume (names)

**Backend (`backend/.env`, server-side):**

```
AUTH_PROVIDER=stub            # flip to "auth0" to cut over (default stays "stub")
AUTH0_DOMAIN=<tenant>.us.auth0.com
AUTH0_AUDIENCE=https://api.caul.cauldron
AUTH0_ISSUER=https://<tenant>.us.auth0.com/     # note trailing slash
AUTH0_JWKS_URL=https://<tenant>.us.auth0.com/.well-known/jwks.json
```

**Frontend (`frontend/.env`, public identifiers — CRA uses the `REACT_APP_` prefix, not `VITE_`):**

```
REACT_APP_AUTH_PROVIDER=stub
REACT_APP_AUTH0_DOMAIN=<tenant>.us.auth0.com
REACT_APP_AUTH0_CLIENT_ID=<client id>
REACT_APP_AUTH0_AUDIENCE=https://api.caul.cauldron
```

None of the SPA values are secrets. There is **no client secret** for this flow.

## 3. Libraries / versions

- Frontend: `@auth0/auth0-react@2.28.2` (Universal Login, Auth Code + PKCE; works with CRA/craco).
- Backend: `PyJWT[crypto]` using `PyJWKClient` for JWKS retrieval + RS256 verification (no hard-coded keys).

## 4. How it maps onto the existing codebase

- **New `Auth0IdentityProvider`** implementing the current `IdentityProvider` interface (`app/auth_provider.py`), selected when `AUTH_PROVIDER=auth0`. Stub remains for `stub`.
- **Backend token verification** (`app/auth0.py`, new): `PyJWKClient` validates signature, `RS256`, `iss`, `aud`, `exp/iat/sub`. Reject ID tokens (must be API access token for our audience).
- **`get_current_user`** (`app/deps.py`) gains an Auth0 path: verify bearer -> map `sub` -> local user -> resolve RBAC from Postgres exactly as today. The stub JWT path is kept behind the flag.
- **Users table:** add a unique `auth0_subject` column; link by **verified email** during migration, then key on `sub` thereafter. Linking is written to the audit chain.
- **MFA:** enabled in the Auth0 tenant (Security -> Multi-factor Auth; prefer WebAuthn/passkeys or TOTP). Our current `Mfa.jsx` stub is replaced by Auth0 Universal Login challenge — no MFA code in our app.
- **Frontend:** wrap the app in `Auth0Provider`; `loginWithRedirect`/`logout`; attach `Authorization: Bearer <access token>` (audience-scoped) on API calls; keep tokens in memory.

## 5. Migration / cutover plan (staging first)

1. Add `auth0_subject` column + migration (no behavior change while `AUTH_PROVIDER=stub`).
2. Stand up the **staging** Auth0 tenant; set the 4 backend + 3 frontend values as staging secrets.
3. Create the ExecOwner user in Auth0 (**sean.gaden@gmail.com**) + any real staff; verify email.
4. Flip `AUTH_PROVIDER=auth0` in **staging** only; run the §6 test matrix.
5. Rollback = flip back to `stub` (zero data loss; stub accounts untouched).
6. Production tenant + cutover only after a separate approval.

## 6. Test matrix (to keep the 100% gate green, with mocked verifier + a dev-tenant token)

valid RS256 -> 200 · expired -> 401 · wrong audience -> 401 · wrong issuer -> 401 · bad signature/unknown `kid` -> 401 · ID-token-as-bearer -> rejected · valid identity w/o local user -> controlled provisioning/403 · valid user w/o permission -> 403 · valid user w/ permission -> 200 · JWKS key rotation refresh · MFA challenge completed vs cancelled · flag `stub` path still fully works (rollback).

## 7. What I need from you before Phase 1B coding

1. **Written approval** to begin Phase 1B (staging-only identity).
2. Decision: do you already have an **Auth0 account/tenant**, or should I include tenant-creation steps?
3. The **staging** values: `AUTH0_DOMAIN`, `AUTH0_CLIENT_ID`, `AUTH0_AUDIENCE` (I derive issuer/JWKS from the domain).
4. **MFA policy** choice (Always / Adaptive / Action-controlled) and preferred factor (WebAuthn/TOTP).
5. Confirm **staging-only** first (recommended) vs. also provisioning a production tenant now.
6. Confirm real users to seed in Auth0 (starting with **sean.gaden@gmail.com** as ExecOwner).

## 8. Cost / ToS note

Auth0 has a free tier, but it is an **external service connection** (outside the Phase 1A `$0`/no-egress boundary). Enabling it is a deliberate spend/ToS decision — hence the explicit approval gate above.

---

**Reminder:** this dossier authorizes nothing. No Auth0 code is wired and no external request is made until you approve §7.